

Annotation Platform Architecture Discussion

Multi-agent 設計整理：評估目前系統是否足以彈性擴展多種 labeling tool、常見 annotation format，以及如何讓不同 Oracle schema、Oracle table、web service、權限與資料欄位的外部系統容易接入。

70%

Format adapter 彈性已有基礎

45%

Labeling tool runtime 還需要 adapter 化

30%

Enterprise integration 尚缺正式 common interface

1. Executive Summary

判斷：目前系統已經從 X-AnyLabeling 專用，演進到支援 LabelMe、X-AnyLabeling、ISAT、COCO、YOLO detection、YOLO segmentation 的第一版多格式架構。核心方向正確：平台內部保留 canonical annotation model，外部格式透過 adapter 進出。

但如果目標是讓不同系統容易對接，目前還不算完整彈性。主要缺口不是再多加一個 exporter，而是需要把 **Format Registry**、**Labeling Tool Adapter**、**External System Connector**、**Schema Mapping Profile**、**Capability Matrix** 與 **Conversion Loss Report** 拉成正式 common interface。

最重要的設計原則是：annotation core 永遠只處理平台 canonical model；所有外部格式差異放在 format adapter；所有工具差異放在 tool adapter；所有客戶系統差異放在 integration connector 與 mapping profile。這樣才能同時支援 LabelMe、ISAT、COCO、YOLO，也能接不同 Oracle schema、REST service、身份權限與資料欄位，而不讓核心流程被客製邏輯污染。

可行性結論： 這個 solution 是可行的，但應採漸進式改造。第一步不要直接做大型 plugin system 或 Oracle 雙向同步，而是先把目前已存在的 adapters、tool launch、service dispatch 收斂到 registry 與 contract。等核心 extension point 穩定後，再把外部系統 connector、mapping profile、export delivery 接上去。

2. Feasibility Review

重新檢視後，我認為這份設計的方向正確，且可以落在目前 repo 架構上。它不是要推翻既有 AnnotationService、MCP、module_006/008/009/012，而是把這些既有入口逐步變成 registry 與 connector 的 façade。

設計項目	可行性	落地方式	主要風險
FormatRegistry	高	把目前 LabelMe、X-AnyLabeling、ISAT、COCO、YOLO adapters 先註冊成 descriptors，再讓 service 透過 registry dispatch。	若 descriptor 太複雜，第一版會拖慢開發；應先支援 import/export/capability/aliases。
ToolRegistry	中高	包住現有 labeling_runtime.py 與 X-AnyLabeling runtime，逐步讓 module_006/012 使用同一套 launch contract。	不同 GUI 工具的 CLI 能力不一致，collect results 需要以 capability 表示，不要假設每個工具都能完整自動化。
ExternalSystemConnector	中	先做 contract、fake connector、file connector，再接 Oracle/REST。核心不要直接 import Oracle driver。	客戶系統差異大，若沒有 profile/mapping versioning，會變成到處加客製 if/else。
IntegrationProfile	高	用 JSON/YAML 描述 system_id、tenant、connector、field mapping、schema mapping、format policy。	profile 欄位要版本化，否則後續 migration 會痛。
Structured ConversionReport	高	延伸既有 ConversionReport，加入 loss summary、field losses、	若 UI 不顯示 loss report，使用者仍會誤以為 export 一定無損。

設計項目	可行性	落地方式	主要風險
		mapping version、dry-run 結果。	
Module migration	中	module_006 已接近 generic； module_008 需從 LabelMe-style session JSON 轉向 canonical bridge；module_009 需分階段拆 X-Any coupling。	module_009 tracking correction 與 X-AnyLabeling 結合較深，不適合一次改完。

建議落地策略：先把 registry 與 contract 建起來，保留既有 public API 與 legacy aliases。也就是外部呼叫者仍可用 `annotation_create_export`、`annotation_prepare_labeling_project`，但內部 dispatch 從 hard-coded `if/elif` 改成 registry。這樣風險最低，也最容易用現有 tests 證明沒有 regression。

3. 設計圖與 Interface 標註

這一章把前面的設計轉成可溝通的圖。重點是讓討論時可以快速回答三件事：資料從哪裡來、標註工具和輸出格式如何分開、各系統要實作哪個 common interface。



- SchemaMappingProfile



Annotation Core

- Dataset / Asset / Schema
- AnnotationSet / Review
- Validation / Provenance



Extension Registries

- FormatRegistry
- ToolRegistry
- ConversionReport



Delivery APIs

- MCP / REST / modules
- Export artifacts
- Push back to systems

Annotation core 是唯一的 system of record。Oracle schema、REST endpoint、LabelMe JSON、YOLO TXT 都只出現在邊界層，不進核心模型。

3.2 Labeling Tool 與 Annotation Format 分離圖

ToolRegistry

- X-AnyLabeling
- LabelMe GUI
- ISAT / ISAT-SAM



Tool Project / Session

- prepare project
- launch GUI
- collect result files



FormatRegistry

- LabelMe JSON
- ISAT JSON
- COCO / YOLO



Canonical Model

- bbox / polygon
- classification
- future mask/keypoint

溝通重點：用 X-AnyLabeling 標註，不代表只能 export X-AnyLabeling；用 ISAT 標註，也可以 export COCO。Tool 是人機互動流程，format 是資料交換契約。

3.3 Import / Export 雙向流程圖

Import

Connector pull task/assets



Field/schema mapping



Format adapter import

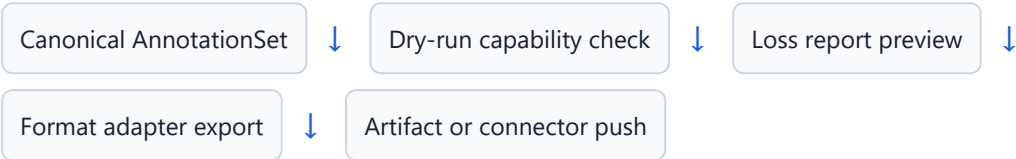


Canonical AnnotationSet



Validate / review

Export



3.4 Common Interface 標註圖

FormatAdapter



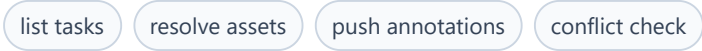
只負責 canonical model 與外部格式互轉。不得連 Oracle、不得處理使用者權限。

LabelingToolAdapter



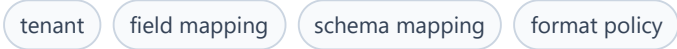
只負責 GUI tool lifecycle。不得決定客戶最終 export 格式，也不得直接回寫外部系統。

ExternalSystemConnector



封裝 Oracle、REST、檔案服務與客戶 API。不得讓 SQL/table 欄位污染 annotation core。

IntegrationProfile



用設定描述每個系統如何對接。新增客戶時優先新增 profile/connector，不修改 core。

3.5 實作 Roadmap 圖

Phase 1

FormatRegistry：把既有 LabelMe、ISAT、COCO、YOLO adapters 註冊化。

Phase 2

ConversionReport：支援 export dry-run 與 loss report preview。

Phase 3

ToolRegistry：收斂 X-AnyLabeling、LabelMe、ISAT 的 launch/prepare。

Phase 4

IntegrationProfile：導入 field/schema/status mapping 與 format policy。

Phase 5

Connectors：先 file/fake，再 REST/Oracle，避免核心綁資料庫。

Phase 6

Orchestration：import/export job、retry、conflict detection。

Phase 7

Module migration：module_006/MCP 先行，module_009 最後分階段。

Phase 8

Contract tests：format/tool/connector/profile 都有共同驗收測試。

4. Multi-Agent Findings

Agent A：目前 repo 架構

`sidecar/python-engine/annotation` 已有 canonical model、format adapters、service API；`mcp/annotation_mcp` 已有 generic tools；OpenSpec 也明確區分 annotation format 與 labeling tool。這是好的基底。

但 `AnnotationService` 仍用 `if/elif` 做 adapter dispatch；`module_008` 內部仍偏 LabelMe-style JSON；`module_009` tracking correction 仍高度 X-AnyLabeling specific；`module_012` 自己做 tool launch dispatch。這些都是下一步要收斂的地方。

Agent B：外部系統整合

不同系統的差異不只在標註格式，也包含 Oracle table/schema、web service API、身份權限、資料 lifecycle、審核流程、回寫策略、錯誤重試與版本衝突。這些不應該進 annotation core，應由 integration connector 與 mapping profile 承接。

Agent C：Format / Export / Import

LabelMe、X-AnyLabeling、ISAT、COCO、YOLO 的資料模型不同。困難點不是能不能寫出檔案，而是轉換時哪些資訊會遺失。例如 polygon 匯出 YOLO detection、bbox 匯出 YOLO segmentation、mask 匯出未支援格式，都應在 conversion report 中清楚呈現。

5. Current Architecture Assessment

面向	目前狀態	評估
Canonical Model	已有 <code>Dataset</code> 、 <code>ImageAsset</code> 、 <code>LabelSchema</code> 、 <code>AnnotationSet</code> 、 <code>Annotation</code> 、bbox、polygon、classification。	方向正確，是多格式支援的核心。
Format Adapter	已有 LabelMe、X-AnyLabeling、ISAT、COCO、YOLO detection、YOLO segmentation。	已有基礎，但還不是 registry/plugin 型。
Service API	已有 <code>prepare_labeling_project</code> 、 <code>import_annotations</code> 、 <code>import_project_labels</code> 、 <code>supported_annotation_formats</code> 、 <code>create_export</code> 。	API 名稱已往 generic 方向走。

面向	目前狀態	評估
Service Dispatch	格式與工具 dispatch 仍散在 <code>services.py</code> 的條件判斷。	格式越多越難維護，應改成 registry 。
Tool Runtime	已有 detect/launch 起點，但 project layout、output collection、single image/project mode 還沒完整 interface 化。	工具擴展性不足。
External Integration	尚未正式抽出 Oracle/REST/schema/identity/mapping connector contract。	若要支援多系統對接，這是最大缺口。

6. Tool 與 Format 必須分開

設計上要明確分開兩個概念：

- **Labeling tool**：可啟動或準備 project 的工具，例如 X-AnyLabeling、LabelMe、ISAT。
- **Annotation format**：檔案與資料交換格式，例如 LabelMe JSON、ISAT JSON、COCO、YOLO detection、YOLO segmentation。

一個工具可能使用某種格式，但不等於格式本身。例如 X-AnyLabeling 大致使用 LabelMe-compatible JSON，但有自己的 launch args、project layout、rectangle 表現、工具行為。Export 時使用者選 COCO 或 YOLO，不代表標註時一定要用 COCO 或 YOLO 工具。

7. 建議目標分層

Layer	責任	不應該知道
Annotation Core	canonical dataset、asset、schema、annotation set、review state、validation、provenance。	Oracle table、REST endpoint、LabelMe/ISAT/YOLO 檔案細節。
Format Adapter	canonical model 與 LabelMe、ISAT、COCO、YOLO、CVAT 等格式互轉。	客戶系統資料庫、權限、任務 lifecycle。

Layer	責任	不應該知道
Labeling Tool Adapter	detect runtime 、prepare project 、launch GUI 、collect output 、描述 tool capabilities 。	Oracle schema 與客戶回寫邏輯 。
Integration Connector	連 Oracle 、REST 、檔案服務、客戶 API ，讀取任務/影像/schema ，回寫標註結果 。	格式轉換細節與 GUI tool lifecycle 。
Mapping/Profile	欄位 mapping 、label taxonomy mapping 、status mapping 、format policy 、tenant policy 。	核心 validation 演算法 。
Orchestration/API	import job 、labeling session 、review job 、export job 、sync/retry/conflict detection 。	個別 adapter 的檔案細節 。

8. Common Interface 建議

8.1 FormatAdapter

```
class FormatAdapter:
    format_id: str

    def descriptor(self) -> FormatDescriptor:
        ...

    def import_file(self, input_path, context) -> ImportResult:
        ...

    def import_directory(self, input_dir, context) -> ImportResult:
        ...

    def export(self, annotation_set, output_dir, context) -> ExportResult:
        ...

    def dry_run_export(self, annotation_set, context) -> ConversionReport:
        ...
```

8.2 LabelingToolAdapter

```
class LabelingToolAdapter:
    tool_id: str

    def detect_runtime(self) -> RuntimeStatus:
        ...

    def prepare_project(self, assets, schema, output_dir, context) -> ProjectResult:
        ...

    def launch(self, project_dir, context) -> LaunchResult:
        ...

    def expected_outputs(self, project_dir, context) -> list[ArtifactRef]:
        ...

    def collect_results(self, project_dir, context) -> CollectedArtifacts:
        ...
```

8.3 ExternalSystemConnector

```
class ExternalSystemConnector:
    system_id: str

    def list_tasks(self, query, page) -> Page[ExternalTask]:
        ...

    def resolve_asset(self, external_task) -> ResolvedAsset:
        ...

    def load_label_schema(self, schema_ref) -> LabelSchema:
        ...

    def push_annotations(self, payloads, mode) -> list[PushResult]:
        ...

    def mark_task_state(self, external_task_id, state, actor) -> StateResult:
        ...

    def check_conflicts(self, payloads) -> list[Conflict]:
        ...
```

8.4 IntegrationProfile

```
{
  "system_id": "customer_a_mes",
  "tenant_id": "fab_01",
  "asset_source": {
    "type": "oracle",
    "connector": "oracle_image_source_v1",
    "query_id": "pending_inspection_images"
  },
  "annotation_sink": {
    "type": "oracle",
    "connector": "oracle_annotation_sink_v1",
    "write_mode": "upsert"
  },
  "schema_source": {
    "type": "web_service",
    "connector": "customer_taxonomy_api_v1"
  },
  "format_policy": {
    "default_labeling_tool": "x-anylabeling",
    "default_export_format": "coco",
    "allowed_export_formats": ["labelme", "isat", "coco", "yolo-detection"]
  },
  "field_mapping": {
    "external_task_id": "TASK_ID",
    "external_asset_id": "IMAGE_ID",
    "image_uri": "IMAGE_PATH",
    "width": "IMG_WIDTH",
    "height": "IMG_HEIGHT"
  },
  "schema_mapping": {
    "label_mappings": {
      "DEFECT_A": "scratch",
      "DEFECT_B": "dent"
    },
    "unknown_label_policy": "warn_and_skip"
  }
}
```

9. Format Registry 與 Capability Matrix

建議將目前散在 service 中的格式判斷搬到 registry。新增格式時只註冊 descriptor 與 adapter，不修改核心 service flow。

```
FormatDescriptor:
  id: "labelme" | "isat" | "coco" | "yolo-detection" | ...
  display_name: "LabelMe JSON"
  aliases: ["xanylabeling", "labelme-style"]
  layout: "single-file" | "directory" | "dataset-file"
  can_import: true
  can_export: true
  supported_geometries:
    bbox: full | approximated | unsupported
    polygon: full | approximated | unsupported
    mask: full | approximated | unsupported
    keypoint: full | approximated | unsupported
    classification: full | approximated | unsupported
  supports_attributes: true | partial | false
  supports_confidence: true | false
  supports_instance_id: true | false
  adapter: FormatAdapter
```

Format	BBox	Polygon	Mask	Keypoint	Attributes	主要風險
LabelMe	Full	Full	Partial/Future	Partial/Future	Partial	不同工具對 shape_type、flags 解讀不同。
X-AnyLabeling	Full	Full	Partial/Future	Partial/Future	Partial	看似 LabelMe，但工具端有自己的慣例。
ISAT	Approx	Full	Future	No	Partial	bbox 可能要轉 polygon/segmentation approximation。
COCO	Full	Full	RLE Future	Future	Partial	category id、image id、RLE、iscrowd、area。
YOLO Detection	Full	No	No	No	No	只保留 class + bbox，polygon/attributes 會遺失。

Format	BBox	Polygon	Mask	Keypoint	Attributes	主要風險
YOLO Segmentation	No/Approx	Full	No	No	No	只支援 polygon rows , attributes 會遺失。
Pascal VOC	Full	No	Separate VOC Segmentation	No	Partial	XML schema , 通常 bbox only。
CVAT	Full	Full	Partial/Full	Partial/Full	Full-ish	task model 較大 , image/video task 差異大。

10. Export 應如何決定 LabelMe / ISAT / COCO / YOLO

Export 不應只看使用者標註時用了哪個工具，而應由三個因素共同決定：

- **用途**：human review、training、customer handoff、model evaluation、archive。
- **目標系統能力**：客戶系統能吃 COCO、YOLO、ISAT、LabelMe 或自訂格式。
- **資料內容**：bbox、polygon、mask、classification、track、attribute 是否會在目標格式中失真。

建議 export API 使用明確 request：

```
{
  "annotation_set_id": "aset_xxx",
  "target_system": "customer_a_mes",
  "purpose": "training",
  "export_format": "coco",
  "delivery": {
    "type": "connector_push",
    "sink": "customer_a_oracle_annotation_sink"
  },
  "options": {
    "include_images": false,
    "class_id_policy": "schema_order",
    "on_loss": "warn"
  }
}
```

若使用者沒有指定 `export_format`，平台可以從 integration profile 的 `format_policy.default_export_format` 選擇。但實際匯出前應做 dry-run capability check，讓使用者知道是否會丟失 polygon、attribute、mask、instance metadata。

11. Import / Export 流程建議

Export flow

使用者或外部系統建立 export request

- > 查 IntegrationProfile 與 FormatRegistry
- > 檢查 CapabilityMatrix
- > 套用 SchemaMappingProfile
- > Dry-run conversion
- > 產生 LossReport preview
- > 使用者或 policy 確認
- > 實際 export artifact
- > 儲存 artifact + conversion report + mapping profile version
- > 透過 AnnotationSinkConnector 回寫外部系統

Import flow

使用者或外部系統指定 import format

- > Registry 找 adapter
- > 解析檔案或資料夾
- > AssetResolver 對應 image
- > SchemaMappingProfile 對應 label/category/class index
- > Geometry normalization
- > Validation
- > 寫入 canonical AnnotationSet
- > 回傳 unmatched / skipped / approximated / unknown labels report

12. Conversion Loss Report 應升級成一級公民

目前已有 `ConversionReport`，但未來應更結構化，讓 UI、API、測試與外部系統都能理解。

```
ConversionReport:
  lossless: false
  source_format: "canonical"
  target_format: "yolo-detection"
  summary:
    exported_annotations: 120
    skipped_annotations: 8
    approximated_annotations: 3
  losses:
    - annotation_id: "ann_001"
      severity: "dropped"
      field: "polygon"
      reason: "YOLO detection does not support polygon"
      suggested_format: "yolo-segmentation"
    - annotation_id: "ann_002"
      severity: "approximated"
      field: "bbox"
      reason: "ISAT stores segmentation; bbox converted to rectangle polygon"
  class_mapping:
    scratch: 0
    dent: 1
  coordinate_transform:
    from: "pixel"
    to: "normalized"
  warnings:
    - "Attributes were not exported to YOLO"
```

13. 不污染 Annotation Core 的做法

- Core model 只放平台共通欄位，不放 `CUSTOMER_A_IMAGE_ID` 這種欄位。
- 外部 primary key 放在 `metadata` 或 `provenance.external_refs`。
- Oracle SQL、REST URL、token、table name 只存在 connector config。
- 客戶欄位轉換寫在 mapping profile，不寫在 annotation service。
- 客戶特殊 JSON 寫成 custom format adapter，不修改既有 LabelMe/COCO adapter。
- 客戶特殊 push 行為寫成 sink connector，不修改 core storage。

- 多 tenant 時，workspace、schema、credential、audit log 應有清楚 boundary。

14. Recommended Target Structure

```
annotation-platform/  
  core/  
    canonical models  
    validation  
    review state  
    provenance  
  
  formats/  
    registry.py  
    contracts.py  
    labelme adapter  
    x-anylabeling adapter  
    isat adapter  
    coco adapter  
    yolo adapter  
    customer custom format adapters  
  
  tools/  
    registry.py  
    contracts.py  
    labelme tool adapter  
    x-anylabeling tool adapter  
    isat tool adapter  
  
  integrations/  
    contracts/  
      common DTOs  
      capability model  
      error model  
      versioning  
    connectors/  
      oracle connector  
      rest connector  
      file connector  
      custom connector  
  profiles/
```

```
customer_a.yaml
customer_b.yaml
mappings/
  field mapping
  schema mapping
  status mapping

orchestration/
  import workflow
  labeling workflow
  review workflow
  export workflow
  sync queue
  retry
  conflict detection

api/
  MCP tools
  REST API
  internal service API
```

15. Test Strategy

Contract tests

- 每個 format adapter 必須能列出 descriptor/capability。
- 每個 adapter 都能 export empty annotation set。
- bbox、polygon、classification 支援子集要有共同測試。
- 遇到 unsupported geometry 不可 silent fail，必須寫 loss report。
- import 後 canonical model 正確。
- class mapping deterministic。

Golden file tests

為 LabelMe、ISAT、COCO、YOLO、Pascal VOC、CVAT 建立小型 golden samples。每次 adapter 修改都比較輸出 JSON/XML/TXT 結構。

Round-trip tests

```
lossless format:
  canonical -> format -> canonical

lossy format:
  canonical -> format -> canonical + expected loss report
```

Integration contract tests

- Oracle connector 用 fake Oracle 或 contract fixture 測 list/pull/push。
- REST connector 用 mock server 測 auth、pagination、retry、conflict。
- mapping profile 測未知 label、版本差異、欄位缺失。

16. Suggested OpenSpec Direction

下一個 OpenSpec 不建議只叫「add another format」。它應該處理 extension mechanism：

```
openspec/changes/annotation-format-registry-and-external-integration/
```

1. 新增 `FormatRegistry` 與 `FormatAdapter` protocol。
2. 新增 `ToolRegistry` 與 `LabelingToolAdapter` protocol。
3. 新增 `IntegrationProfile` schema。
4. 新增 `ExternalSystemConnector` contracts。
5. 新增 schema/field/status mapping policy。
6. 新增 export dry-run 與 structured loss report。
7. 把 module_006/008/009/012 的 tool/format dispatch 收斂到 registry。
8. 補 contract tests、golden tests、round-trip tests、integration connector tests。

17. Final Recommendation

建議：你現在的系統已經有可擴展多格式 labeling 的基礎，但要成為能讓不同系統穩定對接的平台，下一步應該不是繼續在 service 裡加更多 `if/elif`，而是正式建立 common interface：

FormatRegistry 、 ToolRegistry 、 ExternalSystemConnector 、 IntegrationProfile 、 SchemaMappingProfile 與 structured ConversionReport 。

做完這一層之後，未來新增 Label Studio 、 CVAT 、 Pascal VOC 、 客戶自訂 Oracle schema 、 REST web service 、 或自訂 export JSON ，都應該是「新增 adapter/profile/connector」，而不是改 annotation core 。

18. 實作設計：如何落地

下面是建議的實作順序。重點是每一階段都能獨立測試、保留既有 API、避免一次改壞既有 workflow 。

Phase 0：建立 OpenSpec 與 contract acceptance criteria

- 新增 openspec/changes/annotation-format-registry-and-external-integration/ 。
- 定義四組需求：format registry 、 tool registry 、 integration profile 、 external connector 。
- 把「不修改 annotation core 來支援客戶系統」寫成明確 requirement 。
- 把 export dry-run 、 loss report 、 mapping profile version 寫成 acceptance criteria 。

Phase 1：FormatRegistry 先落地，替換 service if/elif

先新增格式 contract，但不要搬動既有 adapter 的檔案位置，降低風險。

```
sidecar/python-engine/annotation/formats/  
__init__.py  
contracts.py      # FormatAdapter, FormatDescriptor, FormatCapabilities  
registry.py       # register(), get(), normalize(), list_supported()  
builtins.py       # 註冊 labelme/x-anylabeling/isat/coco/yolo adapters
```

- 將現有 labelme.py 、 isat.py 、 coco.py 、 yolo_detection.py 、 yolo_segmentation.py 包成 adapter descriptor 。
- AnnotationService.supported_annotation_formats() 改讀 registry 。
- AnnotationService.create_export() 、 import_annotations() 、 import_project_labels() 改由 registry dispatch 。
- 保留 import_xanylabeling_* 與既有 MCP tools，內部改呼叫 registry 。

Phase 2：Structured ConversionReport 與 export dry-run

- 擴充既有 `ConversionReport`，加入 `lossless`、`losses`、`mapping_version`、`summary`。
- 新增 `dry_run_export(annotation_set_id, export_format, options)` service API。
- MCP 增加 `annotation_dry_run_export`，UI 可在真正 export 前顯示 loss preview。
- YOLO detection/segmentation、ISAT bbox approximation、COCO RLE unsupported 都應回報結構化 losses。

Phase 3：ToolRegistry 收斂 project preparation 與 GUI launch

```
sidecar/python-engine/annotation/tools/
__init__.py
contracts.py      # LabelingToolAdapter, ToolDescriptor, RuntimeStatus
registry.py
builtins.py      # x-anylabeling, labelme, isat
```

- 把 `labeling_runtime.py` 包成第一版 tool adapters。
- `prepare_labeling_project()` 改讀 ToolRegistry，並由 tool descriptor 決定預設 output format 與 labels directory。
- module_006 優先使用 ToolRegistry，保留 `xany_dir`、`legacy_mode` 相容欄位。
- module_012 的 `_launch_xany/_launch_labelme/_launch_isat` 後續改成呼叫同一個 tool launch API。

Phase 4：IntegrationProfile 與 SchemaMappingProfile

```
sidecar/python-engine/annotation/integrations/
contracts.py      # ExternalSystemConnector, ExternalTask, PushResult
profiles.py      # load/validate profile JSON or YAML
mappings.py      # field/schema/status mapping
connectors/
  file_connector.py
  rest_connector.py
  oracle_connector.py # later, behind optional dependency
```

- 第一版先做 `file_connector` 與 fake connector 測 contract，不急著接正式 Oracle。
- profile 必須有 `version`、`system_id`、`tenant_id`、`format_policy`、`field_mapping`、`schema_mapping`。
- Oracle SQL、table name、REST URL、credential reference 只能存在 connector config/profile，不可進 core model。

- mapping 預設 policy 建議是 `warn_and_skip` 或 `fail`，不要自動建立未知 label，避免 taxonomy 被污染。

Phase 5：Orchestration job API

等 registry/profile 穩定後，再建立高階 workflow API。這一層讓不同系統用一致流程對接。

```
create_import_job(profile_id, query, options)
create_labeling_session(task_ids, tool_id, options)
dry_run_export_job(annotation_set_id, target_system, purpose, export_format)
create_export_job(annotation_set_id, target_system, purpose, delivery)
get_job_status(job_id)
get_conversion_report(job_id)
```

- 短期可同步執行，長期再接 queue/retry。
- 所有 job 都記錄 profile version、mapping version、format adapter version。
- export job 可輸出 artifact，也可透過 connector push 回 Oracle/REST。

Phase 6：Module migration order

Module	建議順序	原因
module_006	第一批	已經最接近 generic flow，可用來驗證 ToolRegistry + FormatRegistry。
MCP	第一批	外部系統與 agent 會走 MCP/API，必須先穩定 contract。
module_012	第二批	單圖 launch dispatch 可收斂到 ToolRegistry，但風險較小。
module_008	第三批	目前內部仍偏 LabelMe-style session JSON，需要 canonical bridge。
module_009	最後分階段	tracking correction 與 X-AnyLabeling coupling 最深，先保留 XAny automation，再逐步拆 schema/table 欄位與 launcher。

Phase 7：驗證策略

- 每個 format adapter 都跑同一組 contract tests。
- 保留既有 annotation、MCP、module_006/008/009/012 regression tests。
- 新增 dry-run loss report golden tests。
- 新增 fake connector integration tests，確保外部 system profile 不會污染 canonical model。
- Oracle connector 先以 contract fixture 測 SQL mapping，不在核心 test 中要求真實 Oracle 連線。

實作結論：可行的第一個 PR 應該只做 `FormatRegistry`，並讓現有 `service` 行為完全不變；第二個 PR 再做 `ToolRegistry`；第三個 PR 才開始做 `IntegrationProfile` 和 fake connector。這個順序最能保護目前已經支援的 `LabelMe`、`X-AnyLabeling`、`ISAT`、`COCO`、`YOLO` 流程。

19. 如何讓 AI 讀這份文件並安全實作

這份 HTML 已經足夠讓 AI 理解目標架構、邊界、phase order 與主要 interface。但是它應該被當成「架構藍圖」，不要當成一次全部實作的指令。最安全的做法是讓 AI 每次只實作一個 phase，並且先建立 `OpenSpec`、補 unit tests、保留既有 API 相容性，再做程式修改。

不要這樣下指令：「請讀這份 HTML，然後全部實作完。」這會讓修改範圍過大，容易同時碰到 `service`、`MCP`、`module_006`、`module_008`、`module_009`、`module_012`、`Oracle connector` 與前端 workflow，風險太高。

建議這樣下指令：「請閱讀 `docs/ANNOTATION_PLATFORM_ARCHITECTURE_DISCUSSION.html`，依照第 18 章只實作 Phase 1：`FormatRegistry`。請先建立 `OpenSpec change`，再新增/更新 unit tests，最後改 `service dispatch`。保留所有既有 public API、`MCP tools`、`legacy aliases`。不要實作 Phase 2 以後的內容。」

19.1 推薦 AI 實作流程

1. 先讀本文件，特別是第 2 章可行性、第 3 章設計圖、第 18 章實作設計。
2. 建立或更新 `OpenSpec change`，明確寫出本次只做哪一個 phase。
3. 先新增 failing tests 或 contract tests，確認目標行為。
4. 只修改本 phase 需要的最小檔案範圍。
5. 保留現有 API、`MCP tool names`、`legacy aliases` 與 `module behavior`。
6. 跑 focused tests，再跑 `annotation/MCP/module regression tests`。
7. 更新文件與 `OpenSpec tasks`，最後再 commit。

19.2 Phase-by-phase AI Prompt 範本

Phase 1 prompt:

請閱讀 `docs/ANNOTATION_PLATFORM_ARCHITECTURE_DISCUSSION.html`。
請只實作 Phase 1：`FormatRegistry`。

需求：

- 建立 `annotation/formats/contracts.py`、`registry.py`、`builtins.py`。
- 把既有 `LabelMe`、`X-AnyLabeling`、`ISAT`、`COCO`、`YOLO detection`、`YOLO segmentation adapters`。
- `AnnotationService.supported_annotation_formats/create_export/import_annotations/import_annotations`。
- 保留所有既有 `public API`、`MCP tools`、`legacy aliases`。
- 不要實作 `ToolRegistry`、`IntegrationProfile`、`ExternalSystemConnector`。
- 先建立 `OpenSpec change` 與 `unit tests`，再修改程式。
- 最後跑 `annotation/MCP/module regression tests`。

Phase 2 prompt:

請閱讀 `docs/ANNOTATION_PLATFORM_ARCHITECTURE_DISCUSSION.html`。

請只實作 Phase 2: Structured ConversionReport 與 export dry-run。

需求：

- 擴充既有 `ConversionReport`，支援 `lossless`、`summary`、`losses`、`mapping_version`。
- 新增 `dry_run_export service API` 與 `MCP tool`。
- 讓 `YOLO/ISAT/COCO unsupported` 或 `approximated conversion` 產生結構化 `loss report`。
- 不要修改 `ToolRegistry` 或 `ExternalSystemConnector`。
- 保留現有 `export` 行為與既有 `conversion_report.json` 相容性。

Phase 3 prompt:

請閱讀 `docs/ANNOTATION_PLATFORM_ARCHITECTURE_DISCUSSION.html`。

請只實作 Phase 3: ToolRegistry。

需求：

- 建立 `annotation/tools/contracts.py`、`registry.py`、`builtins.py`。
- 將 `X-AnyLabeling`、`LabelMe`、`ISAT runtime detect/launch/prepare project` 包成 `tool adapter`。
- `prepare_labeling_project` 與 `launch_labeling_project` 改讀 `ToolRegistry`。
- `module_006` 優先接 `ToolRegistry`，保留 `xany_dir`、`legacy_mode`、`xany_*` 相容欄位。
- 不要實作 `IntegrationProfile` 或 `Oracle/REST connector`。

19.3 AI 實作 Guardrails

- 每次只做一個 phase，不跨 phase 修改。
- 不得把 `Oracle table name`、`REST URL`、`客戶欄位名稱`寫進 `annotation core`。
- 不得刪除現有 `X-AnyLabeling compatibility API`。
- 不得讓 `module_009 tracking correction` 在第一輪 `registry refactor` 中大改。
- 新增格式時必須註冊 `capability matrix`，並補 `conversion/loss tests`。
- 任何有損 `export` 必須能在 `ConversionReport` 中被看見。
- 每個 phase 完成後要有獨立 `commit`，方便 `rollback`。

19.4 AI 完成條件

檢查項目	完成標準
OpenSpec	有 proposal/design/tasks/spec，且本 phase tasks 已勾選。
Tests	新增 phase 對應 tests，既有 annotation/MCP/module regression tests pass。
Compatibility	既有 service API、MCP tools、legacy aliases、module flows 不破壞。
Documentation	README、OpenSpec、本討論文件或相關 handoff docs 已同步。
Scope control	沒有實作本 phase 以外的大型功能，例如 Phase 1 不做 Oracle connector。

總結： AI 可以讀這份 HTML 來實作，但請把它當成分階段工程藍圖。每次交給 AI 的任務都應明確說：「只做某一個 phase」、「保留相容性」、「先測試後修改」、「不要跨 phase」。這樣成功率最高，也最能保護目前已經能運作的多格式 labeling 流程。

20. Codebase 現狀 Gap Analysis（Multi-Agent 掃描結果）

以下為對 `sidecar/python-engine/annotation/`、`mcp/annotation_mcp/`、`scripts/module_006/008/009/012/` 進行完整程式碼審查後的對比報告。

20.1 已存在且符合設計方向

項目	檔案	符合度	說明
Canonical Models	<code>annotation/core/models.py</code>	100%	Dataset / ImageAsset / LabelSchema / AnnotationSet / Annotation 完整，含 BBoxGeometry、PolygonGeometry、ClassificationValue discriminated union，及完整 state enum。
Storage Ports	<code>annotation/storage/ports.py</code>	100%	MetadataStore / ArtifactStore 抽象協議完整，SQLiteMetadataStore + LocalArtifactStore 正確實作。

項目	檔案	符合度	說明
Validation Engine	<code>annotation/core/validation.py</code>	100%	結構化 ValidationIssue，包含 schema mismatch、unknown label、geometry constraint、bbox bounds、polygon validity 等六類規則。
State Machine	<code>annotation/core/states.py</code>	100%	VALID_ANNOTATION_SET_TRANSITIONS 強制狀態機，非法轉移拋出 InvalidStateTransition。
MCP Tools	<code>mcp/annotation_mcp/server.py</code>	100%	61 個 <code>annotation_*</code> 前綴工具，JSON 序列化統一在 handlers.py 處理。
Format Adapters	<code>annotation/adapters/</code>	70%	LabelMe、X-AnyLabeling、ISAT、COCO、YOLO detection/segmentation 皆存在，但無 Registry，改用 if/elif hard-coded dispatch。
Tool Runtime	<code>annotation/adapters/labeling_runtime.py</code>	80%	detect/launch 起點已有，但 if/elif dispatch (line 31–39)。

20.2 已存在但需 Refactor 的部分（具體位置）

services.py 中 5 處 if/elif dispatch（最高優先）：

- Tool dispatch (lines 221–228)：`if normalized in {"x-anylabeling", "labelme"} ... elif normalized == "isat"`
- Export format dispatch (lines 358–370)：每新增一種格式需在此加一個 elif
- Import format dispatch (lines 275–290、304–320)：同樣問題
- `labeling_runtime.py` lines 31–39：detect tool 的 if/elif

每新增格式或工具都必須改動這些地方，且容易遺漏。應全部改成 Registry dispatch。

module_012 / module_006 中的重複與混入邏輯：

- `012_process.py` line 162：工具白名單 `{"x-anylabeling", "labelme", "isat"}` 硬編碼，應改為從 ToolRegistry 動態取得。
- `_normalize_tool()`、`_normalize_format()` 在 `006_process.py`、`services.py`、`labeling_runtime.py` 三處重複定義，應統一到 shared constants。

- `012_process.py` 直接 import manifest DB（動態載入 `_manifest_db.py`），職責混亂，難以 mock 測試。

module_009 完全綁定 X-AnyLabeling（`_xany_launcher.py`）： `open_labeling_tool()` 在非 XAny 時直接回傳 `unsupported error`。聲稱多工具支援，但實作只有 XAny，需要抽象 `LabelingToolLauncher` interface。

20.3 完全缺失、需要新建

缺失項目	影響	建議位置
AdapterRegistry	高	<code>annotation/formats/registry.py</code> + <code>annotation/tools/registry.py</code>
Job Queue（目前是 stub）	高	<code>handlers.py</code> line 227–231 現在 hardcode 回傳 "succeeded" — 大型 export 無法非同步追蹤
Audit Trail	中	<code>annotation/core/audit.py</code> （設計文件提到但完全未實作）
Manifest ↔ Annotation Adapter 層	中	<code>annotation/adapters/manifest_adapter.py</code> ， <code>module_012/006</code> 目前各自直接查 manifest DB
Coordinate Space Transform 追蹤	中	<code>ConversionReport</code> 有 <code>coordinate_transform</code> 欄位但未填寫；COCO/YOLO import 時未記錄 <code>pixel→normalized</code> 轉換
ExternalSystemConnector + IntegrationProfile	高	<code>annotation/integrations/</code> （整個目錄完全不存在）

20.4 程式碼中發現的設計疑問

AnnotationSet vs Task ID 混用： `services.py` 的 `get_task(task_id)` (line 107) 實際回傳的是 `AnnotationSet`，MCP tool 叫 `annotation_create_task` 但內部呼叫 `create_annotation_set()`。命名不一致會讓外部呼叫者混淆。建議決定：Task 是 `AnnotationSet` 的 view，還是獨立實體？

ConversionReport.lossless 邏輯反轉： 初始值為 `True`，呼叫 `mark_loss()` 時設為 `False`。但如果沒有填入 `dropped_fields`，`lossless=False` 但沒有具體損失資訊，語意模糊。

SQLite 缺乏事務隔離設定： `sqlite_store.py` lines 23–26 沒有設定 `journal_mode=WAL` 或 `isolation_level`，多並發寫入（Streamlit autorefresh + button click）有 dirty read 風險。

AI 預測邏輯在 Output Page： `012_output.py` lines ~75–100 包含 YOLO 模型載入與推論，應抽到 `AnnotationService.predict_with_model()`，Output page 不應含 ML 邏輯。

21. 外部整合設計疑問與建議（深度評論）

以下為從「外部系統整合工程師」角度，針對設計文件提出的具體疑問。每條都附上風險與建議。

21.1 ExternalSystemConnector 介面疑問

Q1：list_tasks 的分頁機制 — offset-based vs cursor-based

目前介面簽名 `list_tasks(query, page)` 隱含 offset-based pagination。Oracle 大量資料時，offset-based 有嚴重 skew：若 import 進行中有新任務插入，同一筆可能被重複或跳過。更常見的是 DBA 提供 View，無 cursor token 概念。

建議： `page` 改為 `PaginationToken` (opaque string)，各 Connector 自行決定實作方式 (keyset / stored procedure cursor / offset)。需明確規定「任務列表的穩定排序欄位」作為前提條件。

Q2：resolve_asset — 影像在 Oracle 中以 BLOB 存放時如何處理？

設計假設影像有可直接存取的 file path，但企業環境常見 BLOB 欄位、SMB share 無法掛載、或 Token-based 短效 URL。若無 streaming/temp download 機制，記憶體壓力大（工業相機影像動輒 50MB+），且暫存檔若無清理會洩漏磁碟空間。

建議： `ResolvedAsset` 改為 Union type，支援 `LocalFilePath` | `RemoteURL` | `BlobStream` | `ContentAddressedRef`，含 `t1` 欄位 (Token URL 過期時間)。Connector 負責 BLOB 下載，暫存路徑由平台統一注入。

Q3：check_conflicts — conflict 語意完全未定義

並發寫入衝突、版本衝突、語意衝突 (label 不一致) 是三種完全不同的問題。用 check-then-act 模式防並發衝突是 TOCTOU 問題，根本無法保護。

建議：明確定義 `Conflict` type system (含 `conflict_type` 和 `resolution_hint`)，並發保護由 DB-level locking 處理，`check_conflicts` 只做版本號對比查詢。

Q4：push_annotations 的 mode 語意不完整

insert-only / upsert / replace-all / append-labels / patch 行為差異巨大。Upsert 的 natural key (「以哪個 key 判斷已存在」) 完全未定義。

建議：`mode` 改為結構化物件，Connector 在 CapabilityMatrix 中聲明支援哪些 mode，job 建立時就 fail-fast，不要等 push 時才報錯。

Q5：Oracle Batch Commit — all-or-nothing 支援

若外部系統要求一個 task 下所有 annotation 必須 all-or-nothing (同時寫 header + detail 兩張表)，Orchestration 層如果逐筆調用 push，中途失敗會造成 Oracle 資料不一致。Retry 若無幂等 key，可能造成重複寫入。

建議：Connector 需聲明 `supports_batch_commit` 和 `idempotency_key_field`，Orchestration 層尊重 Connector 的 transaction boundary 宣告。

Q6：錯誤重試策略缺失

哪些錯誤是 Transient (retry with backoff)、哪些是 Fatal (dead-letter + 告警)、哪些是 BusinessLogic (人工介入) 完全未定義。

建議：Exception 要帶分類標記，Orchestration 根據標記決定處理路徑；沒有 dead-letter 機制的話，失敗 job 可能靜默消失。

21.2 IntegrationProfile 設計疑問

Q7：Profile 缺乏版本欄位 — Audit 追溯失效

如果 `field_mapping` 被修改，過去的 export artifact 無法追溯是用哪個版本 mapping 產生。Regulatory 環境 (品管記錄) 可能要求完整重現能力。

建議：Profile 必須有 `version` + `effective_from` + `checksum`；每次 export job 的 provenance 記錄必須儲存「執行時的 profile snapshot」，而非只是參照可能被修改的 profile ID。

Q8：query_id 安全性 — SQL Injection 風險

"query_id": "pending_inspection_images" 若直接代入 SQL 執行，存在 SQL injection 風險。即使是內部系統，若 profile 被惡意修改，後果嚴重。

建議：`query_id` 必須對應 allowlist registry (管理員預先登記合法的 query handle)，Connector 只能引用已登記的 handle。

Q9：Credential 管理完全缺失（最高優先）

Profile 裡完全沒有 Oracle connection string、密碼、REST API token 的存放設計。若寫在 profile JSON 中，任何能讀取 profile 的人都能看到明文 credential。

建議：獨立 CredentialStore 機制，profile 只存 secret reference（`"credential_ref": "vault://fab_01/oracle_prod"`），Connector 透過 `CredentialProvider` 介面取得認證，不從 profile 直接讀取。這是實作第一個 **Oracle Connector** 之前必須設計完成的前提條件。

Q10：同一客戶多個 Oracle Instance（prod/staging）無法區分

`system_id: "customer_a_mes"` 無法區分 prod/staging，但兩個環境的 field_mapping 往往不同（staging schema 常落後一個版本）。

建議：Profile 加入 `environment` 欄位，執行 job 時強制校驗 annotation platform 自身環境與 profile target environment 一致，否則拒絕執行。

Q11：Schema Source 不可用時的 Fallback 機制缺失

`schema_source: {type: "web_service"}` 若 web service 暫時維護中，整個標注 workflow 是否全面停擺？若 schema 動態變化（今天與昨天不同），進行中的標注使用過期 label 集合。

建議：必須定義 schema TTL、不可用時的 fallback 行為（用 cached version 繼續 vs. fail），以及 schema immutability 保證：AnnotationSet 一旦開始使用某版 schema 就 pin 住，直到完成。

21.3 Schema Mapping 疑問

Q12：warn_and_skip 的 skip 粒度未定義

跳過「這個 annotation」vs 跳過「整張圖所有 annotation」vs 跳過「整個 batch job」，損失範圍差異極大。`warn` 的 warning 要送到哪裡？沒有追蹤機制的 warning 等於沒有。

建議：明確定義 skip 粒度，並且必須產生結構化 `SkipReport`（含每個被跳過的 annotation ID + 原因），讓使用者在 confirm export 之前就能看到，而非事後從 log 挖。

Q13：樹狀 Label Taxonomy 的 Mapping 無法表達

Flat key-value mapping 無法處理「缺陷 > 外觀缺陷 > 刮痕」這樣的階層 label。Export 時父節點 category 資訊會丟失，外部系統若期望 hierarchical 結構則無法填充。

建議：Schema Mapping 支援 path-based mapping（`"缺陷/外觀缺陷/刮痕"`：

`{"external_class": "SCRATCH", "external_parent": "SURFACE"}`），CapabilityMatrix 加入「是否支援 hierarchical label」維度。

Q14：Label 為數字 ID 時的型別轉換問題

JSON key 必然是字串，Oracle 中 `DEFECT_CLASS_ID = 7` (integer) vs `"7"` (string) 可能靜默 mapping 失敗，尤其是有 leading zero 或 compound key 時。

建議： Profile 加入 `label_id_type: "string" | "integer" | "compound"`，Connector 層做明確型別轉換。

Q15：Schema Mapping 版本更新 — 歷史 AnnotationSet 如何處理

客戶把 `"DEFECT_A"` 廢棄、改為 `"DEFECT_A1"` / `"DEFECT_A2"` 時，已完成的 AnnotationSet 要 retroactive 更新？freeze？dual-label？這是業務決策，不能留到實作時再定。

建議： 必須在設計階段確定 migration 策略，且 AnnotationSet 必須有 `schema_version_ref` 欄位，SchemaStore 必須是 immutable（不能刪除已被引用的 schema 版本）。

21.4 多 Tenant 與 Audit**Q16：Canonical Model 是否有 `tenant_id`？資料隔離層級未定義**

設計文件提到 multi-tenant 但 canonical model 欄位沒有列出 `tenant_id`。若靠 application logic 做隔離（WHERE clause 加 tenant filter），一個遺漏的 query 就可能造成跨 tenant 資料洩漏。

建議： 必須明確決定 tenancy model：Row-level isolation（`tenant_id` + RLS）、Schema-level isolation（每 tenant 獨立 DB schema），或 Database-level isolation。這個決策影響所有 CRUD 操作設計，**必須在第一行 DB migration 之前確定。**

Q17：Audit Log 設計完全缺失

誰在什麼時間用哪個 profile 版本對哪個 AnnotationSet 做了什麼操作（export/import/delete）、結果成功還是失敗——無從查詢。Regulatory 環境（ISO、FDA 21 CFR Part 11）可能強制要求完整 audit trail。

建議： 獨立 AuditLog 機制，每個 job 完成（無論成功失敗）都寫入，且必須 append-only（任何 API 都不能刪除或修改）。

21.5 Onboarding、Callback、Event-Driven**Q18：新客戶 Onboarding 流程缺乏定義**

文件沒有說明第一次對接 Oracle 時的 onboarding 步驟、需向客戶 DBA 索取哪些資訊、需要哪些帳號權限，以及如何驗證 connector 設定正確。

建議： 設計 ConnectorHealthCheck 介面，讓 Management UI 可以逐步驗證：① 連線 ping、②

list_tasks 冒煙測試、③ resolve_asset 可達性、④ push_annotations dry-run。詳見第 22 章 Diagram 3。

Q19：外部系統反向通知機制缺失

目前是單向：annotation platform 拉取任務、推送結果。若 MES 品管員人工審核後認為標注有誤，要如何通知平台重新標注？

建議：定義 callback/webhook 機制，或在 list_tasks 查詢中包含「因 rework 被退回」的任務狀態，ExternalTask 加入 rework_reason 欄位。

Q20：Event-Driven 外部系統支援缺失

設計假設 annotation platform 主動 polling。若外部系統用 Kafka / RabbitMQ / Oracle AQ 發布事件，現有架構無法配合，且 polling 對 Oracle 造成查詢壓力。

建議：Connector 在 CapabilityMatrix 中聲明工作模式（pull vs push），Orchestration 根據 connector 宣告選擇驅動方式。

Q21：Orchestration 的觸發機制未定義

是 cron job、API call trigger、還是外事件？沒定義的話各 Connector 可能各自實作 scheduler，造成競爭條件。

建議：文件化觸發來源、最小觸發間隔、concurrent job 限制（同一 profile 同時只能有一個 running import job？），及 job 狀態機（queued → running → succeeded/failed/partial）。

21.6 三個最重要的實作前決策

決策一（最緊急）：Tenancy Model 與 Canonical Model 資料隔離層級

Row-level vs Schema-level vs Database-level isolation。一旦上線後再改，代價極高（data migration + 全面 API 審查）。必須在寫第一行 DB migration 之前確定。

決策二（最緊急）：Credential 管理架構與 Secret Rotation 策略

IntegrationProfile 目前完全沒有 credential 設計，但所有外部連線都需要認證。影響 Connector 介面、onboarding 流程、Security Review。不能等到實作第一個 Oracle Connector 時才臨時決定。

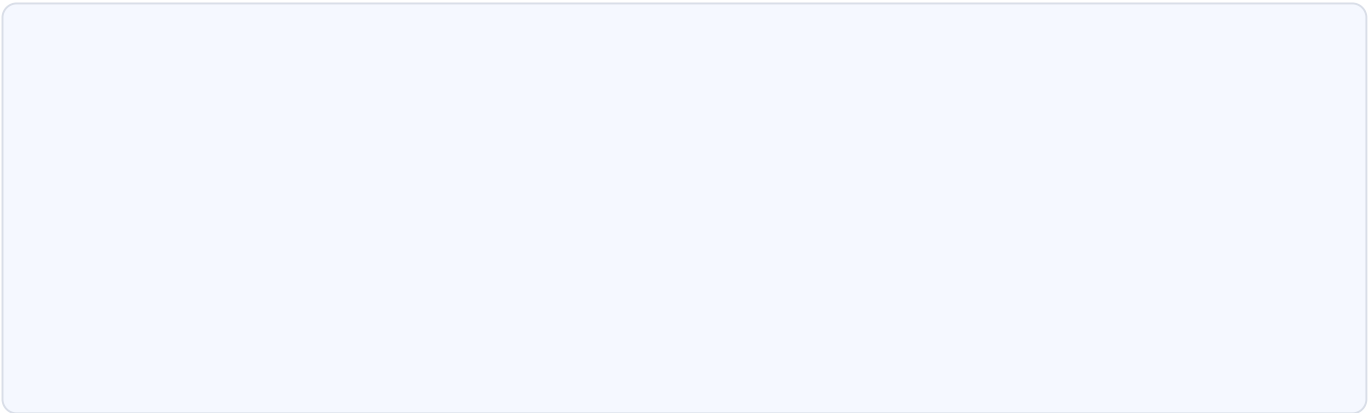
決策三：Schema Mapping 版本不可變性與 AnnotationSet Schema Pin 機制

決定了 canonical model 必須記錄哪些元數據（`schema_version_ref` 欄位），及 SchemaStore 的 immutability 設計。若不在設計階段確定，migration script 將難以撰寫。

23. 簡易版架構總覽圖（供討論用）

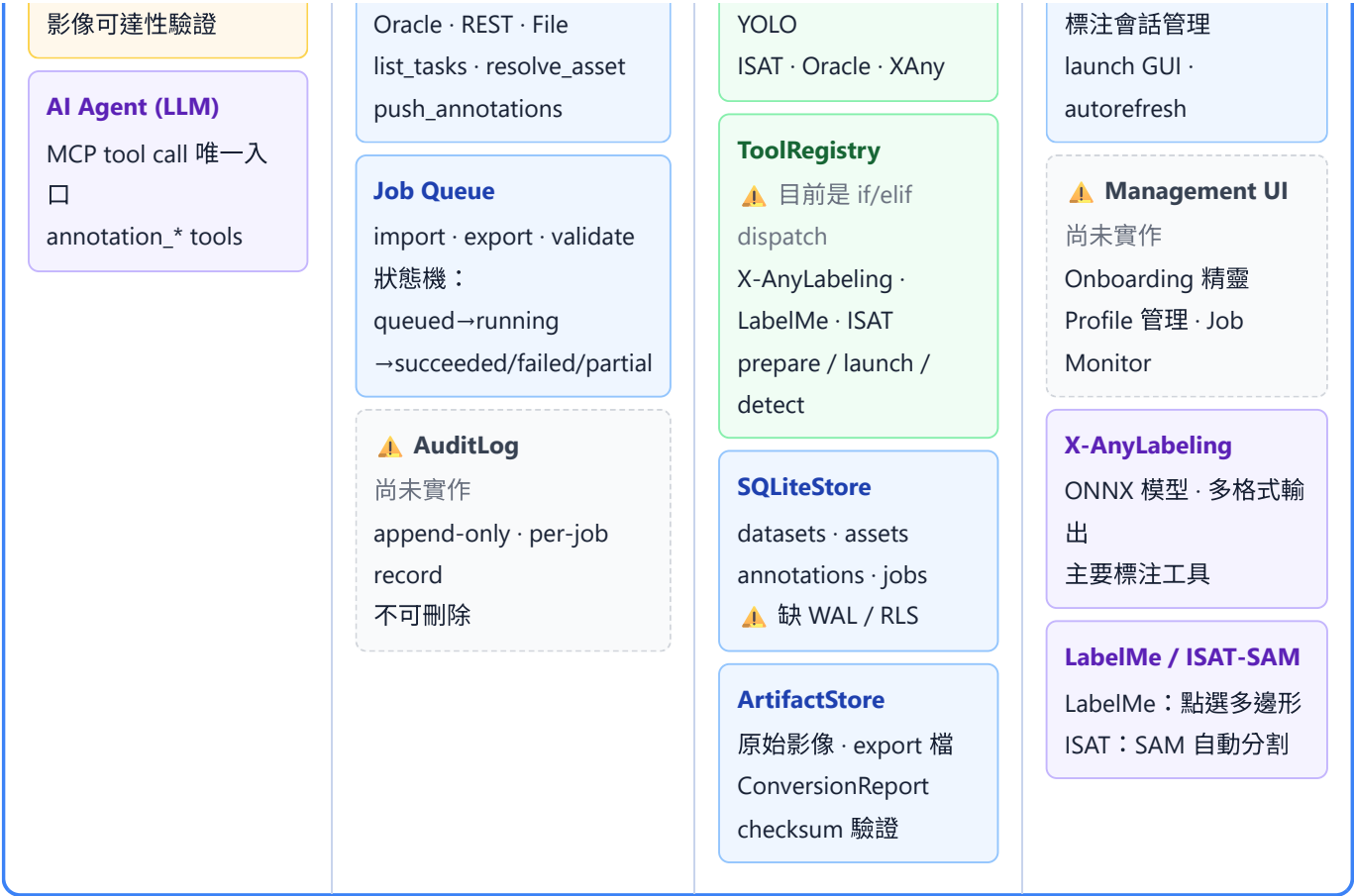
以下圖表以「由外而內」的方向呈現系統邊界與各層關係。  灰色虛線框 標示目前尚未實作、需優先決策的元件。

23.1 系統層次概覽（左→右 = 外部→整合→核心→工具）



23.2 各層元件對照說明

外部入口	Integration + Orchestration	Annotation Core + Storage	UI + 標注工具
Oracle MES V_CIM_TASKS view V_CIM_LABELS view write_annotations API REST API JSON / XML · Token auth 需定義 pagination contract File Share UNC / S3 · PresignedURL	IntegrationProfile DSN · credentials field_mapping · schema_mapping format_policy · capability  CredentialStore 尚未實作 Secret encryption · rotation vault integration Connector	AnnotationService create_dataset · ingest_assets upsert_annotations · create_export validate_annotations FormatRegistry  目前是 if/elif dispatch LabelMe · COCO ·	Streamlit module_006 準備 Labeling 專案 ToolRegistry → prepare_project Streamlit module_009 追蹤標注進度 AnnotationSet 狀態查詢 Streamlit module_012



23.3 Multi-Agent 審查發現：圖表覆蓋缺口

三大 Critical 問題在 Section 22 Sequence Diagrams 中的覆蓋度不足：

- **Tenancy Model (Q16)**：三張圖完全沒有 `tenant_id`，dataset / credential 都缺 tenant context，無法反映此設計決策的影響。
- **Credential 管理 (Q9)**：Diagram 3 的 `create_integration_profile` 直接帶 `credential_ref`，但 CredentialStore 作為獨立元件未出現在圖中——credential 存入、加密、取出路徑不可見。
- **Schema Pin (Q11、Q15)**：Diagram 1 的 `create_schema` 回傳 `schema_id` 後，AnnotationSet 從未出現 `schema_version_ref` 的讀取與鎖定步驟——schema pin 發生在哪一個呼叫完全看不出來。

建議：在討論完三大決策後，補畫一張「Credential 取用 + Schema Pin」的 mini sequence diagram。

22. Sequence Diagrams — 平台與外部服務 Handshaking

以下三張 Sequence Diagram 涵蓋標注平台與外部服務之間的完整 handshaking 流程，以及其他參與方（DBA、標注員、審核員、AI Agent）需要如何配合。

22.1 完整主流程：Import → Labeling → Review → Export

觸發者：外部 Oracle MES，透過 webhook 或 polling 通知有新批次任務。

關鍵風險點：resolve_asset 失敗（影像路徑失效）、GUI process crash 需 watchdog timeout、LossReport 有 critical loss 需人工介入調整 schema mapping、Oracle write_annotations 失敗需重試。

Oracle 需配合：暴露 V_CIM_TASKS view、V_CIM_LABELS view、write_annotations API，並確認 CIM Platform Server IP 在 Oracle whitelist 內。

```
sequenceDiagram
    participant Oracle as ExternalSystem
    participant Conn as Connector
    participant Core as AnnotationCore
    participant FmtAdp as FormatAdapter
    participant ToolAdp as ToolAdapter
    participant GUI as LabelingTool
    participant Orch as Orchestration
    participant StreamUI as Annotator UI
    participant ReviewUI as Reviewer UI

    Oracle->>Conn: webhook / polling → new_task_available(batch_id)
    Conn->>Oracle: list_tasks(batch_id, pagination_token)
    Oracle-->>Conn: TaskList[{task_id, image_uri, metadata}]

    loop For each task
        Conn->>Oracle: resolve_asset(image_uri)
        Oracle-->>Conn: FilePath / PresignedURL
        alt Asset not accessible
            Conn-->>Orch: ingest_error(task_id, reason)
            Orch-->>StreamUI: notify – import failed
        end
    end

    Conn->>Core: create_dataset(name, source=oracle, batch_id)
    Core-->>Conn: dataset_id
    Conn->>Core: ingest_assets(dataset_id, assets[])
    Core-->>Conn: asset_ids[]

    Conn->>Oracle: load_label_schema(batch_id)
    Oracle-->>Conn: RawLabelSchema
    Conn->>FmtAdp: convert_schema(RawLabelSchema)
    FmtAdp-->>Conn: CanonicalSchema
    Conn->>Core: create_schema(dataset_id, CanonicalSchema)
```

```

Core-->>Conn: schema_id
Conn-->>Orch: dispatch_job(import_complete, dataset_id)

StreamUI->>Orch: start_labeling_job(dataset_id, tool=xanylabeling)
Orch->>ToolAdp: prepare_project(dataset_id, schema_id)
ToolAdp->>Core: fetch_assets(dataset_id)
Core-->>ToolAdp: AssetList
ToolAdp->>Core: fetch_schema(schema_id)
Core-->>ToolAdp: CanonicalSchema
ToolAdp->>FmtAdp: schema_to_tool_format(CanonicalSchema)
FmtAdp-->>ToolAdp: XAnyLabelingConfig
ToolAdp-->>Orch: project_ready(project_dir)

Orch->>ToolAdp: launch_gui(project_dir)
ToolAdp-)GUI: spawn_process(xanylabeling.exe)
Note over GUI: 標注員手動標注 (非同步)
GUI-)ToolAdp: process_exit(exit_code=0)

loop For each annotated asset
    ToolAdp->>FmtAdp: convert_annotations(raw_json, schema_id)
    FmtAdp-->>ToolAdp: CanonicalAnnotationSet
    ToolAdp->>Core: upsert_annotations(asset_id, annotations)
    Core-->>ToolAdp: annotation_id
end

Orch->>Core: validate_annotations(dataset_id, schema_id)
alt Validation failed
    Core-->>Orch: ValidationReport(errors)
    Orch-->>StreamUI: show_validation_errors
else Validation passed
    Core-->>Orch: ValidationReport(ok)
    Orch->>Core: set_status(dataset_id, pending_review)
end

ReviewUI->>Core: list_pending_review(dataset_id)
Core-->>ReviewUI: AnnotationList
loop For each asset
    ReviewUI->>Core: review_task(asset_id, verdict)
    alt Rejected
        Core->>Orch: dispatch_rework(asset_id)
        Orch-->>StreamUI: notify – rework required
    end
end

Orch->>FmtAdp: export_dry_run(dataset_id, oracle_annotation)
FmtAdp-->>Orch: LossReport
Orch-->>ReviewUI: show_loss_report(LossReport)
alt Critical losses
    ReviewUI->>Orch: abort_export
else Acceptable
    ReviewUI->>Orch: confirm_export

```

```

end

Orch->>FmtAdp: export(dataset_id, oracle_annotation)
FmtAdp-->>Orch: ExportPayload
Orch->>Conn: push_results(dataset_id, ExportPayload)
Conn->>Oracle: write_annotations(batch_id, ExportPayload)
alt Write success
    Oracle-->>Conn: ack(rows_written)
    Conn->>Core: set_status(dataset_id, exported)
else Write failed
    Oracle-->>Conn: error(code, message)
    Conn->>Orch: export_error(dataset_id, message)
    Orch-->>ReviewUI: notify - export failed, retry?
end

```

22.2 AI Agent 透過 MCP 進行 Annotation 的流程

觸發者：外部 AI Agent（LLM pipeline / CI bot / Claude instance），完全以 MCP tool call 驅動，不需要人工介入。

關鍵非同步點：ingest_assets 和 create_export 回傳 job_id，Agent 需主動輪詢 get_job_status 直到完成。

主要風險：upsert_annotations schema mismatch（Agent 推論格式與 schema 不符）、export job timeout、checksum 不一致需重新觸發。

```

sequenceDiagram
    participant Agent as AI Agent
    participant MCP as MCP Layer
    participant Core as AnnotationCore
    participant Orch as Orchestration
    participant FmtAdp as FormatAdapter
    participant Store as ArtifactStore

    Agent->>MCP: annotation_create_dataset({name, tags})
    MCP->>Core: create_dataset(name, source=ai_agent)
    alt Name conflict
        Core-->>MCP: error(409, name_exists)
        MCP-->>Agent: tool_error
    else OK
        Core-->>MCP: {dataset_id}
        MCP-->>Agent: tool_result({dataset_id})
    end

    Agent->>MCP: annotation_create_schema({dataset_id, schema_def})
    MCP->>Core: create_schema(dataset_id, CanonicalSchema)

```

```

Core-->>MCP: {schema_id}
MCP-->>Agent: tool_result({schema_id})

Agent-->>MCP: annotation_ingest_assets({dataset_id, asset_uris[]})
MCP-->>Orch: dispatch_job(ingest, dataset_id, asset_uris)
MCP-->>Agent: tool_result({job_id, status=queued})

Orch-)Store: fetch_assets(asset_uris[])
Store-->>Orch: asset_bytes[]
Orch-->>Core: ingest_assets(dataset_id, assets)
Core-->>Orch: asset_ids[]
Orch-->>Core: set_job_status(job_id, completed)

loop Poll until done
  Agent-->>MCP: annotation_get_job_status({job_id})
  MCP-->>Core: get_job_status(job_id)
  Core-->>MCP: {status, ingested_count}
  MCP-->>Agent: tool_result({status})
end

loop For each asset
  Agent-->>Agent: run_inference(asset_uri)
  Agent-->>MCP: annotation_upsert_annotations({asset_id, annotations[], source=ai_model})
  MCP-->>Core: validate_annotations(annotations, schema_id)
  alt Schema mismatch
    Core-->>MCP: error(422, mismatch_details)
    MCP-->>Agent: tool_error
    Note over Agent: Fix format and retry
  else Valid
    Core-->>Core: upsert_annotations(asset_id, annotations)
    Core-->>MCP: {annotation_id}
    MCP-->>Agent: tool_result({annotation_id})
  end
end

opt Submit for human review
  Agent-->>MCP: annotation_submit_for_review({dataset_id})
  MCP-->>Core: set_status(dataset_id, pending_review)
  MCP-->>Agent: tool_result({status=pending_review})
end

Agent-->>MCP: annotation_create_export({dataset_id, format=coco_json})
MCP-->>Orch: dispatch_job(export, dataset_id, format)
MCP-->>Agent: tool_result({export_job_id, status=queued})

Orch-->>FmtAdp: export(dataset_id, coco_json)
FmtAdp-->>Core: fetch_all_annotations(dataset_id)
Core-->>FmtAdp: CanonicalAnnotationSets
FmtAdp-->>Store: write_export_file(coco.json)
Store-->>FmtAdp: export_uri
FmtAdp-->>Core: set_job_status(export_job_id, completed, {export_uri})

```

```

loop Poll until done
    Agent->>MCP: annotation_get_job_status({job_id: export_job_id})
    MCP-->>Agent: tool_result({status, progress_pct})
end

Agent->>MCP: annotation_get_export({export_job_id})
MCP->>Core: get_export(export_job_id)
Core-->>MCP: {download_url, checksum}
MCP-->>Agent: tool_result({download_url, checksum})

Agent->>Store: GET download_url
Store-->>Agent: coco.json
Agent->>Agent: verify_checksum(file, checksum)
alt Checksum mismatch
    Agent->>MCP: annotation_create_export(...)
    Note over Agent: Re-trigger export
else OK
    Note over Agent: Export complete
end

```

22.3 新客戶 Oracle Onboarding 流程

觸發者：Platform Admin，手動透過 Management UI 驅動，每個階段都需要 Admin 確認才進下一步。

最常見卡關點：Oracle TNS 連線失敗（防火牆封鎖）、V_CIM_TASKS view 不存在、影像路徑無法存取、oracle_schema 沒有對應 FormatAdapter。

Oracle DBA 需配合：建立 V_CIM_TASKS / V_CIM_LABELS view、提供 write_annotations 接受端、將 CIM Server IP 加入 whitelist、若用 File Share 需授予 read-only 存取權。

sequenceDiagram

```

participant Admin as Platform Admin
participant Profile as ProfileStore
participant Conn as Connector
participant Oracle as Customer Oracle
participant FmtAdp as FormatAdapter
participant Core as AnnotationCore
participant Orch as Orchestration

```

Note over Admin: 客戶提供 Oracle DSN, credentials, schema info

```

Admin->>Profile: create_integration_profile({customer_id, dsn, username, credential_ref, imag
Profile->>Profile: encrypt_credentials()
Profile-->>Admin: {profile_id, status=created}

```

```
Admin->>Conn: test_connection(profile_id)
Conn->>Profile: load_profile(profile_id)
Profile-->>Conn: IntegrationProfile
Conn->>Oracle: SELECT 1 FROM DUAL
alt Connection failed
    Oracle-->>Conn: ORA-12541 / timeout
    Conn-->>Admin: {status=failed, error}
    Note over Admin: Fix DSN / credential and retry
else OK
    Oracle-->>Conn: OK
    Conn-->>Admin: {status=connected, oracle_version, latency_ms}
end

Admin->>Conn: smoke_test_list_tasks(profile_id, limit=5)
Conn->>Oracle: SELECT * FROM V_CIM_TASKS WHERE ROWNUM <= 5
alt View not found
    Oracle-->>Conn: ORA-00942 table/view does not exist
    Conn-->>Admin: {status=failed, suggestion=create V_CIM_TASKS}
    Note over Admin: Notify customer DBA to create view
else OK
    Oracle-->>Conn: TaskRows[5]
    Conn-->>Admin: {status=ok, sample_tasks, column_map_suggestion}
end

Admin->>Conn: test_resolve_asset(profile_id, sample_image_uri)
alt UNC File Share
    Conn->>Conn: check UNC path accessible
    alt UNC not reachable
        Conn-->>Admin: {status=failed, error=unc_unreachable}
    else OK
        Conn-->>Admin: {status=ok, file_size_kb}
    end
else S3 / HTTP
    Conn->>Oracle: get_presigned_url(image_uri)
    Oracle-->>Conn: presigned_url
    Conn->>Conn: HTTP HEAD presigned_url
    alt 403 / 404
        Conn-->>Admin: {status=failed, http_code}
    else 200
        Conn-->>Admin: {status=ok, content_length}
    end
end

Admin->>Conn: load_label_schema(profile_id)
Conn->>Oracle: SELECT * FROM V_CIM_LABELS
Oracle-->>Conn: RawLabelSchema
Conn->>FmtAdp: convert_schema(RawLabelSchema)
alt No adapter for oracle_schema
    FmtAdp-->>Conn: error(no_adapter)
    Conn-->>Admin: {status=failed, suggestion=register OracleSchemaAdapter}
```



```

    Note over Admin: Developer implements and registers adapter
else OK
    FmtAdp-->>Conn: CanonicalSchema (draft)
    Conn-->>Admin: {status=ok, canonical_schema_preview, unmapped_fields[]}
end

opt Fix unmapped fields
    Admin->>Conn: set_schema_mapping(profile_id, mappings)
    Conn->>Profile: save_schema_mapping(profile_id, mappings)
    Conn->>FmtAdp: re_convert_schema(RawLabelSchema, mappings)
    FmtAdp-->>Conn: CanonicalSchema (updated)
    Conn-->>Admin: {status=ok, remaining_unmapped[]}
end

Admin->>Core: create_schema(CanonicalSchema, source=onboarding)
Core-->>Admin: {schema_id}
Admin->>Profile: link_schema(profile_id, schema_id)
Profile-->>Admin: {profile_updated}

Admin->>Orch: dry_run_import(profile_id, sample_limit=10)
Orch->>Conn: list_tasks(profile_id, limit=10)
Conn->>Oracle: SELECT * FROM V_CIM_TASKS WHERE ROWNUM <= 10
Oracle-->>Conn: TaskRows[10]
loop For each sample task
    Conn->>Oracle: resolve_asset(task.image_uri)
    Oracle-->>Conn: resolved_path
    Conn->>Core: ingest_asset(dry_run=true, asset_uri)
    Core-->>Conn: {would_create: asset_id, conflicts[]}
    opt Has existing annotations
        Conn->>FmtAdp: convert_annotations(ExistingAnnotations, schema_id)
        alt Conversion errors
            Conn->>Orch: log_dry_run_error(task_id, errors)
        else OK
            Conn->>Orch: log_dry_run_ok(task_id, annotation_count)
        end
    end
end
Orch-->>Admin: DryRunReport{total_sampled, would_ingest, errors[], estimated_count}

alt DryRunReport has errors
    Note over Admin: Review errors, fix mapping, re-run dry_run
else Clean
    Admin->>Profile: set_profile_status(profile_id, active)
    Profile-->>Admin: {status=active, activated_at}
    Note over Admin: Onboarding complete – Connector 開始正式排程
end

```

